

Contrato técnico do piloto

Fonte e limites

iMC é a única fonte da verdade. Fluxo: GET iMC, GET NetBox, comparação, relatório, PATCH autorizado. Alteram-se apenas descrição e VLAN de interfaces existentes. Nenhum objeto é criado ou excluído; nenhum dispositivo é atualizado.

A allowlist rígida, verificada na comparação e no transporte NetBox, contém exclusivamente:

```
ALLOWED_INTERFACE_PATCH_FIELDS = {  
    "description", "mode", "untagged_vlan", "tagged_vlans",  
}
```

mode só muda para representar a VLAN confirmada. Campos desconhecidos bloqueiam o payload inteiro. Não há atualização de enabled, status, up/down, velocidade, tipo, IP, MAC, site, rack, comentários, tags ou custom fields.

Os clientes bloqueiam métodos antes da rede. A sessão também valida requisições diretas e preparadas. PATCH só pode atingir uma interface individual.

Redirecionamentos não são seguidos. Código legado fica em `imc/legacy/netbox_client.py.disabled`, com bloqueio explícito, fora dos imports e sem habilitação por ambiente.

Correspondência e normalização

Dispositivo: nome exato `sysName`, ou campo `label/name` quando o anterior não estiver presente. Nome vazio não tem fallback por IP. Interface: nome exato `ifName`, `name` ou `ifDescription`, nesta ordem por presença de campo. Não se expande abreviação nem se associa por posição na lista.

O nome técnico `ifDescription` não é usado como descrição. A descrição usa `ifAlias`, `description` ou `ifDesc`, nesta ordem por presença.

- None/null ou string vazia explícitos: descrição vazia, podendo limpar a descrição existente.
- Campo ausente ou tipo inesperado: preservar a descrição NetBox e registrar aviso.
- Remover somente espaços no início/fim; preservar maiúsculas, acentos e espaços internos.
- Resposta inválida ou erro de parsing da coleção: preservar os objetos envolvidos.

Dispositivos/interfaces duplicados no iMC e correspondências duplicadas no NetBox são bloqueados. A interface retornada deve pertencer ao dispositivo consultado.

Dispositivo ausente no NetBox é descarte esperado: registrado pelo nome, sem consultar interfaces, sem criação e sem aumentar o contador de erros.

VLANs

VIDs válidos: 1 a 4094; nenhum VID é fixado ou filtrado como regra operacional. VID não é o ID interno NetBox: a resolução acontece antes da comparação. Texto numérico, objetos contendo VID, ordem e duplicatas são normalizados.

A consulta a portvlan precisa completar com sucesso. Access exige nome de modo/ifIndex inequívoco nos dados disponíveis e uma única VLAN igual ao PVID; falha em trunk/hybrid é registrada, mas não bloqueia esse access confirmado. Trunk exige classificação completa e PVID confirmado na associação; esse PVID representa a VLAN nativa e as demais VLANs são tagged. Essa semântica ainda exige validação na instalação real do iMC.

Hybrid é reconhecido, mas bloqueado enquanto os dados disponíveis não distinguirem inequivocamente VLANs tagged e untagged. O PVID sozinho não é suficiente. Também não se representa trunk sem PVID confirmado, tagged-all ou múltiplas VLANs untagged por inferência.

VID ausente ou duplicado no contexto escolhido bloqueia a proposta inteira da interface, inclusive descrição. NETBOX_VLAN_GROUP_ID restringe a busca e valida o grupo retornado. Sem configuração de grupo, exige VID único global. A seleção de grupo deve refletir o contexto real, sem inferência automática.

Resposta vazia, timeout, autenticação inválida, coleção incompleta, modo desconhecido ou parsing inválido preservam toda a interface. Não há suporte a remoção de todas as associações por ausência de VLAN no iMC.

Uma lista tagged pode ser esvaziada quando uma configuração access completa confirma que a interface possui exclusivamente o PVID. Uma associação nativa pode ser substituída por outra confirmada. Essas transições são comparadas antes do PATCH.

Endpoints ativos

Serviço	Método	Endpoint relativo
---	---	---
iMC	GET	/imcrs/plat/res/device
iMC	GET	/imcrs/plat/res/device/{device_id}/interface
iMC	GET	/imcrs/vlan/portvlan?devId=...&ifIndex=...
iMC	GET	/imcrs/vlan/access?devId=...
iMC	GET	/imcrs/vlan/trunk?devId=...
iMC	GET	/imcrs/vlan/hybrid?devId=...
NetBox	GET	/api/dcim/devices/?name=...
NetBox	GET	/api/dcim/interfaces/?device_id=...&name=...
NetBox	GET	/api/ipam/vlans/?vid=...
NetBox	PATCH	/api/dcim/interfaces/{id}/

iMC pagina com start/size até uma página vazia, inclusive após páginas curtas; NetBox usa offset/limit. Endpoints que ignoram paginação ou repetem páginas falham com segurança. XML usa coleções list com os elementos device, interface, vlan, accessIf, trunkIf e hybridIf; JSON aceita coleções homônimas ou list. Formatos diferentes geram pendência, não associação inferida. Validar esse contrato na versão do laboratório

antes de aplicar.

Execução

Dry-run padrão consulta tudo, normaliza, compara, valida, apresenta valores anteriores/propostos e não envia PATCH. Interface igual é contabilizada como já sincronizada.

Antes da aplicação há resumo detalhado conforme o README. A CLI principal exige `--device NOME` ou `--all-devices` junto de `--apply`. A entrada `aplicar_switch.py` exige um nome e confirmação SIM; `sincronizar_producao.py` seleciona todo o inventário e oferece dry-run padrão. A aplicação não interativa em produção exige `--apply --non-interactive`.

Antes de cada PATCH, nova leitura verifica ID e snapshot da interface. Se mudou, o PATCH é bloqueado. Isso reduz conflitos, mas não oferece transação atômica entre GET e PATCH. Uma alteração concorrente após a nova leitura ainda é possível.

Falha em uma interface não interrompe as demais. Logs apresentam resultado individual e resumo final; falhas retornam código diferente de zero. Código 0: execução sem erros; 1: pendências/falhas no processamento; 2: configuração, lock ou falha global. Cancelamento interativo registrado não envia PATCH.

Depois de PATCH, GET verifica ID e campos propostos: applied só aumenta quando confirmados. Timeout ou desconexão após envio exige reconciliação por GET, sem repetição automática. accepted indica resposta HTTP de sucesso; uncertain identifica envio incerto; unconfirmed registra tentativas cuja alteração não pôde ser confirmada.

Logs e concorrência

Logs em `logs/pilot.log`, com data/hora UTC, UUID, commit e indicação de alterações locais, modo, propostas e resultados. Rotação: 5 MB por arquivo, cinco backups. Diretório 0700, arquivos criados com umask 077. Não registrar corpos completos de resposta, senhas, tokens, cookies ou cabeçalhos de autenticação.

O filtro central mascara credenciais conhecidas e padrões de segredo, inclusive URLs com usuário/senha. A descrição pode conter texto livre: restringir acesso aos logs e nunca versioná-los.

Python usa flock no arquivo fixo `.pilot.lock` da raiz. O wrapper usa também `.wrapper.lock`. Bloqueio cobre execução direta e wrapper na mesma cópia do repositório; cópias distintas exigem coordenação operacional. Não remover arquivos de lock durante execução.

Execução manual única, sem loop. Instalador de cron está desativado. Agendamento somente após validação e por um único mecanismo.